

Module 14 – Containerization using Docker

1. Introduction to Docker

What is Docker?

Docker is an open-source platform that enables developers to **build, package, distribute, and run applications inside lightweight containers.**

A **container** packages an application together with its dependencies, libraries, runtime, and configuration files, ensuring that it runs consistently across different environments.

Docker follows the principle:

"Build once, run anywhere."

What is Containerization?

Containerization is the process of packaging an application and all its dependencies into a single, portable container.

Unlike virtual machines, containers share the host operating system kernel, making them lightweight and fast.

Containers vs Virtual Machines

Container	Virtual Machine
Shares host OS kernel	Has its own guest operating system
Lightweight	Heavier
Starts within seconds	Takes several minutes to start
Uses fewer system resources	Requires more CPU and memory
Highly portable	Less portable

Benefits of Docker

- Platform-independent application deployment.
- Fast application startup.
- Lightweight compared to virtual machines.
- Consistent development, testing, and production environments.
- Easy scalability.
- Simplified dependency management.
- Supports DevOps and CI/CD pipelines.

2. Docker Commands

Docker is primarily managed through the **Docker Command Line Interface (CLI)**.

Frequently Used Docker Commands

Command	Purpose
docker --version	Display Docker version
docker pull <image>	Download an image from Docker Hub
docker images	List available images
docker run <image>	Create and start a container
docker ps	List running containers
docker ps -a	List all containers
docker stop <container>	Stop a running container
docker start <container>	Start a stopped container
docker restart <container>	Restart a container
docker rm <container>	Remove a container
docker rmi <image>	Remove an image
docker exec -it <container> bash	Open an interactive shell inside a container
docker logs <container>	View container logs
docker inspect <container>	Display detailed container information

3. Docker Run

What is docker run?

The docker run command creates and starts a new container from a Docker image.

General syntax:

```
docker run [OPTIONS] IMAGE [COMMAND]
```

Running a Container

```
docker run nginx
```

Downloads the image (if not already available) and starts the container.

Running a Named Container

```
docker run --name my-nginx nginx
```

Assigns the container the name my-nginx.

Running in Detached Mode

Detached mode runs the container in the background.

```
docker run -d nginx
```

The terminal becomes available immediately while the container continues running.

Running Interactively

```
docker run -it ubuntu bash
```

Options:

- -i – Interactive mode.
- -t – Allocate a terminal.

Useful for testing and debugging.

Publishing Ports

Allows access to services running inside the container.

```
docker run -p 8080:80 nginx
```

Here:

- Host Port → **8080**
- Container Port → **80**

Access using:

`http://localhost:8080`

Automatically Remove Container

```
docker run --rm ubuntu
```

The container is automatically deleted after it exits.

Useful for temporary tasks.

Running a Custom Command

```
docker run ubuntu ls
```

Instead of starting the default application, Docker executes the `ls` command.

4. Docker Images

What is a Docker Image?

A **Docker Image** is a read-only template used to create Docker containers.

It contains:

- Application code
- Runtime
- Libraries
- Environment variables
- Configuration files

Containers are created from images.

Image Layers

Docker images consist of multiple **layers**.

Each instruction in a Dockerfile creates a new layer.

Advantages:

- Faster downloads.
- Efficient storage.

- Layer reuse.
- Better caching during builds.

Container Layer

When a container starts, Docker adds a **writable container layer** above the image.

- Image layers → Read-only.
- Container layer → Read/Write.

All runtime changes are stored in this writable layer.

Parent Image

A **Parent Image** is the image used as the starting point for creating another image.

Example:

```
FROM ubuntu
```

Ubuntu becomes the parent image.

Base Image

A **Base Image** is the initial image upon which all subsequent layers are built.

Examples:

- Ubuntu
- Alpine Linux
- Debian

Docker Manifest

A Docker Manifest contains metadata describing an image, including:

- Image layers
- Supported operating systems
- CPU architectures
- Image tags

Container Registry

A **Container Registry** stores Docker images.

Examples:

- Docker Hub
- GitHub Container Registry
- Google Container Registry

- Amazon Elastic Container Registry (ECR)

Container Repository

A **Repository** is a collection of different versions (tags) of the same image.

Example:

```
nginx:latest  
nginx:1.25  
nginx:stable
```

Creating Docker Images

Interactive Method

1. Run a container.
2. Make changes inside the container.
3. Save the container as a new image using docker commit.

This method is suitable for experimentation but is not recommended for production.

Dockerfile Method

A **Dockerfile** is a text file containing instructions to automatically build a Docker image.

Example:

```
FROM openjdk:17  
COPY app.jar app.jar  
ENTRYPOINT ["java","-jar","app.jar"]
```

Build the image:

```
docker build -t my-app.
```

Docker Build Context

The **Build Context** is the directory sent to the Docker daemon during the build process.

It includes:

- Dockerfile
- Source code
- Configuration files
- Other required resources

5. Docker Compose

What is Docker Compose?

Docker Compose is a tool for defining and managing **multi-container Docker applications**.

Instead of running multiple docker run commands, all services are described in a single **YAML configuration file**.

Benefits

- Simplifies multi-container applications.
- Easy service management.
- Single command deployment.
- Consistent environment configuration.
- Supports networking and shared volumes.

Docker Compose File

Docker Compose uses:

docker-compose.yml

or

compose.yaml

Example:

```
version: "3"
```

```
services:
```

```
  web:
```

```
    image: nginx
```

```
  database:
```

```
    image: mysql
```

Common Docker Compose Commands

Command	Purpose
docker compose up	Start all services
docker compose down	Stop all services
docker compose ps	List running services
docker compose logs	View service logs
docker compose build	Build images
docker compose restart	Restart services

6. Docker Engine

What is Docker Engine?

Docker Engine is the core runtime responsible for building, running, and managing Docker containers.

It consists of three primary components.

Docker CLI

The **Docker Command Line Interface (CLI)** allows users to interact with Docker.

Example:

```
docker run nginx
```

The CLI sends commands to the Docker daemon.

Docker Daemon

The **Docker Daemon (dockerd)** is a background service responsible for:

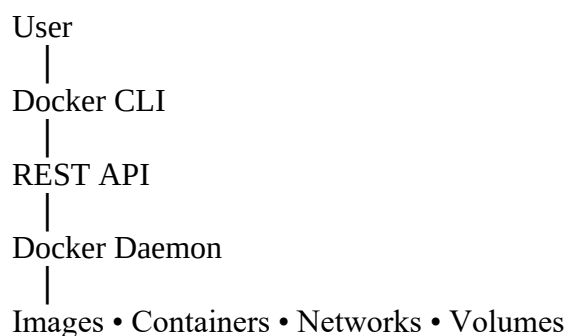
- Building images.
- Running containers.
- Managing storage.
- Managing networks.
- Handling container lifecycle.

REST API

The Docker CLI communicates with the Docker Daemon through the Docker REST API.

Third-party tools can also use this API to automate Docker operations.

Docker Engine Architecture



7. Docker Storage

Containers are temporary by default.

If a container is removed, its data is lost unless persistent storage is used.

Storage Drivers

Storage drivers manage how Docker stores image layers and writable container layers.

Examples include:

- overlay2 (default on most Linux distributions)
- aufs
- btrfs
- zfs

Data Volumes

Volumes provide persistent storage independent of a container's lifecycle.

Benefits:

- Data survives container deletion.
- Easy data sharing between containers.
- Better performance.
- Simplified backups.

Creating a Volume

```
docker volume create mydata
```

Listing Volumes

```
docker volume ls
```

Inspecting a Volume

```
docker volume inspect mydata
```

Using a Volume

```
docker run -v mydata:/app/data nginx
```

The volume is mounted inside the container.

8. Docker Networking

Docker automatically creates virtual networks that allow containers to communicate.

Default Networks

Docker creates three default network types:

Bridge

Default network for standalone containers.

Host

Container shares the host's network stack.

None

Container has no networking.

Listing Networks

`docker network ls`

Inspecting a Network

`docker network inspect bridge`

Displays connected containers and network configuration.

Creating a Network

`docker network create mynetwork`

Containers on the same network can communicate using container names.

Connecting a Container

`docker run --network=mynetwork nginx`

9. Container Orchestration

What is Container Orchestration?

Container Orchestration is the automated management of containerized applications across multiple machines.

It handles:

- Deployment
- Scaling
- Networking
- Monitoring

- Load balancing
- High availability

Why Do We Need Container Orchestration?

Managing a few containers manually is straightforward, but large-scale applications may involve hundreds or thousands of containers.

Orchestration automates tasks such as:

- Starting containers.
- Replacing failed containers.
- Scaling services.
- Managing updates.
- Distributing workloads efficiently.

Benefits of Container Orchestration

- Automated deployment.
- Automatic scaling.
- High availability.
- Self-healing of failed containers.
- Efficient resource utilization.
- Simplified management of distributed applications.

Kubernetes

What is Kubernetes?

Kubernetes (K8s) is the most widely used open-source container orchestration platform.

Originally developed by Google, it automates the deployment, scaling, and management of containerized applications.

Key Features

- Automatic container scheduling.
- Self-healing.
- Load balancing.
- Rolling updates.
- Automatic scaling.
- Service discovery.
- Secret and configuration management.

Docker vs Kubernetes

Docker	Kubernetes
--------	------------

Platform for building and running containers	Platform for orchestrating containers
Creates and manages containers	Manages clusters of containers
Suitable for single-host deployments	Designed for multi-node, distributed deployments
Focuses on container lifecycle	Focuses on application deployment, scaling, and management

Docker and Kubernetes are complementary technologies. Docker creates and runs containers, while Kubernetes manages containers at scale.

Best Practices

- Use lightweight base images (e.g., Alpine Linux) whenever possible.
- Build images using Dockerfiles instead of the interactive method.
- Keep Docker images small by removing unnecessary packages and files.
- Store persistent data in Docker volumes rather than inside containers.
- Use Docker Compose to manage multi-container applications.
- Organize related containers within custom Docker networks.
- Avoid running containers with unnecessary root privileges.
- Regularly update base images to incorporate security patches.
- Use Kubernetes or another orchestration platform for production environments requiring scalability and high availability.

Quick Revision Points

- **Docker** is a containerization platform that packages applications and their dependencies into lightweight, portable containers.
- A **Docker Image** is a read-only template used to create containers, while a **Container** is a running instance of an image.
- Common Docker commands include `docker run`, `docker ps`, `docker stop`, `docker rm`, `docker images`, `docker rmi`, `docker pull`, and `docker exec`.
- The `docker run` command supports options such as `--name`, `-d` (detached mode), `-it` (interactive mode), `-p` (port mapping), and `--rm` (automatic removal after execution).
- Docker images are built from **Dockerfiles**, where each instruction creates a new image layer.
- **Docker Compose** uses a YAML configuration file to define and manage multi-container applications with commands such as `docker compose up` and `docker compose down`.
- The **Docker Engine** consists of the **Docker CLI**, **REST API**, and **Docker Daemon**, which together build, run, and manage containers.
- **Docker Volumes** provide persistent storage that remains available even after containers are removed.
- Docker networking includes **Bridge**, **Host**, and **None** networks, and custom networks can be created for container communication.
- **Container Orchestration** automates deployment, scaling, and management of containers, with **Kubernetes** being the most widely used orchestration platform.